Data frame kitchen

1 Data acquistion

1.1 Reading and writing local files

```
using DataFrames
using CSV

df = CSV.read("source.tsv", DataFrame)
CSV.write("filename.tsv", df; sep = "\t")
```

```
using DataFrames
using Excel
```

1.2 Remotely-hosted files

```
using CSV
using DataFrames
penguins = CSV.read(
    download("https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2020/2
    DataFrame,
    missingstring = "NA"
);
```

1.3 Data repositories

1.3.1 The Paleobiology Database (PaleoDB or PBDB)

```julia
using DataFrames
using PaleobiologyDB

df = pbdb_occurrences(base_name = "Smilodon"; show = "full", vocab = "pbdb")
```

2 Preliminary assessment

2.1 Basic inspection

```julia
using DataFrames
using PaleobiologyDB


size(df)
names(df)
describe(df)
```

2.2 Extracting rows

```julia
using DataFrames
using PaleobiologyDB
taxon_names_df = pbdb_taxa(name = "Carnivora")
families = filter(row -> row.accepted_rank == "family", taxon_names_df) ①
families = subset(taxon_names_df, :accepted_rank => ByRow(==("family"))) ②
families = taxon_names_df[taxon_names_df.accepted_rank .== "family", :] ③
```

① Row-wise function (`DataFrameRow` iteration)
② Column-aware, DataFrames-native transformation
③ Boolean mask indexing (vectorized)

3 Working with `GroupedDataFrame` objects

3.1 Counting the number of groups in a dataset

To count the number of groups, as defined by any arbitrary criteria, e.g. a particular field value, we can use the `groupby` function to split the `DataFrame` into a `GroupedDataFrame`, which can be thought of as a vector of sub-DataFrame's representing a partition of the original `DataFrame`.

```
gdf = groupby(df, :family)
```

The *length* of the `GroupedDataFrame` object, `length(::GroupedDataFrame)`, gives the number of groups.

```
length(gdf)
```